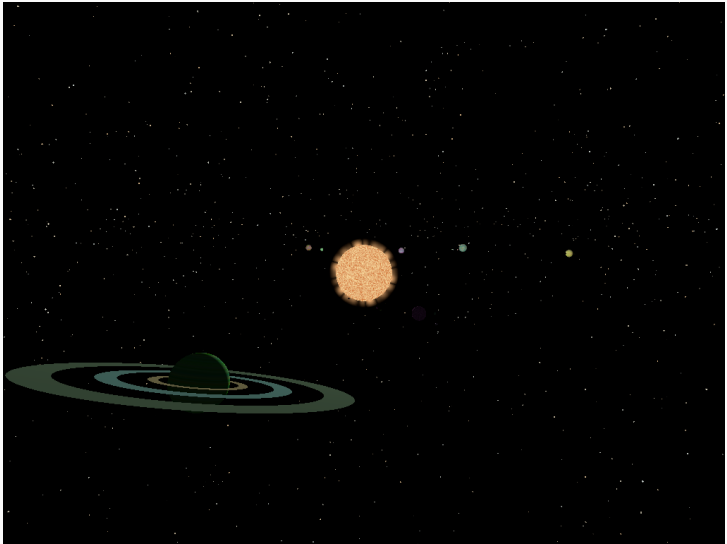


Компьютерная графика

Домашнее задание 1: Планетная система

2026



- Сцена с Солнцем, планетами, и звёздами
- Вокруг некоторых планет есть спутники или кольца
- Планеты вращаются вокруг Солнца, спутники – вокруг планет
- Звёзды неподвижны и “бесконечно далеки”
- Планеты и Солнце крутятся вокруг своей оси и как-нибудь раскрашены
- Цвета у Солнца меняются анимированно, вокруг Солнца – фейковое сияние

- Камера должна достаточно свободно двигаться
- Идеально: WASD – двигаемся вперёд/назад/влево/вправо, движение мыши – вращаем камеру
- Удобно хранить 3D координаты самой камеры + два угла поворота (надо разобраться, вокруг каких осей)
- В начале программы сделайте `app.grab_mouse(true)` – это не даст курсору мыши выйти за пределы окна

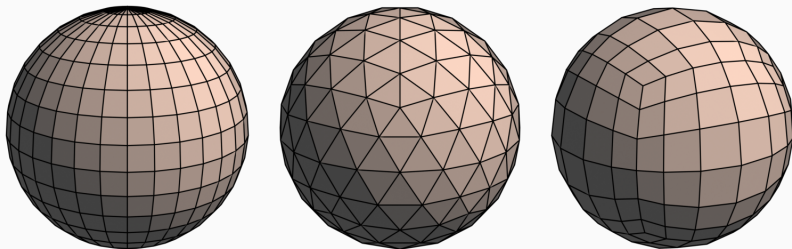
Солнце и планеты

- Нужно сгенерировать кодом 3D модель сферы, вариантов много – UV-sphere, icosphere, subdivided cube
- Главное – чтобы она была с индексами, без дырок, и без дублирующихся вершин (*manifold mesh*)
- На все планеты и Солнце – один буфер с 3D моделью сферы, но у каждого свой буфер с цветами вершин (как-нибудь их сгенерируйте)
- Шейдер Солнца просто выводит цвета как есть, а шейдер планет использует освещение по формуле

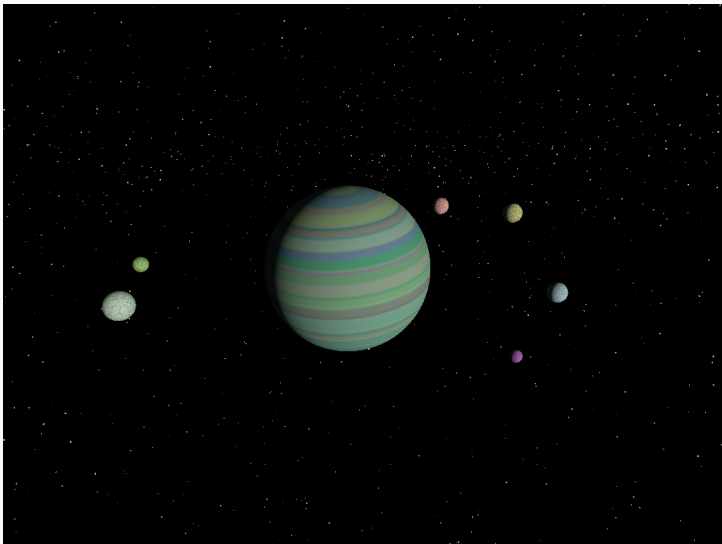
```
// world_position - позиция в 3D пространстве
// считаем, что Солнце находится в начале координат
color * max(0.0, dot(normal, -normalize(world_position)))
```

- Цвета Солнца как-нибудь анимируются на CPU и каждый кадр загружаются на GPU

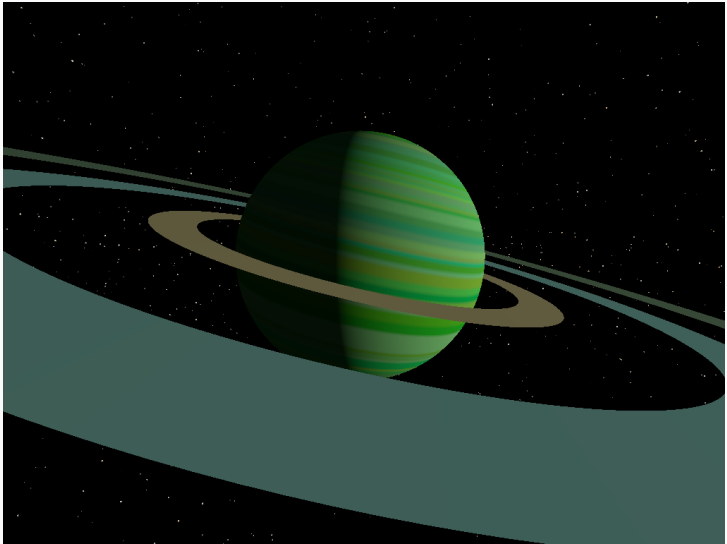
3D модель сферы



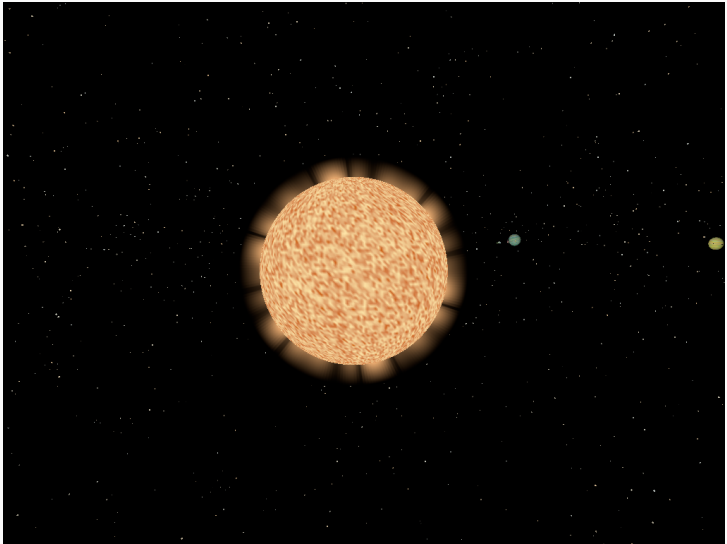
- Планеты движутся вокруг Солнца и вращаются вокруг своей оси (ось может быть наклонена)
- Спутники движутся вокруг своих планет и вращаются вокруг своей оси
- Солнце тоже вращается вокруг своей оси
- Все эти повороты, сдвиги, и масштабирования должны делаться одной матрицей `model`



- Для колец нужно сгенерировать отдельную 3D модель “кольца”
- Она будет односторонней (без внутренней), и будет неправильно освещаться с “обратной” стороны
- Для этого есть полезный флаг во фрагментном шейдере `@builtin(front_facing) : bool` – если он равен `false`, домножим вектор нормали на -1

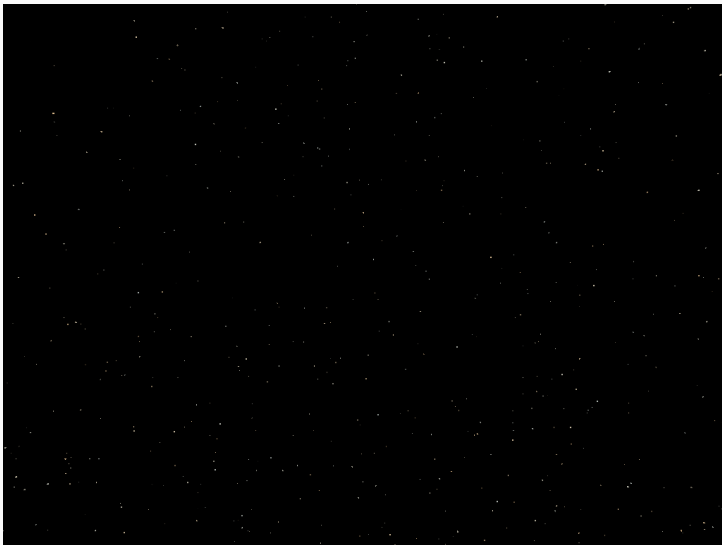


- Для свечения Солнца можно использовать ту же 3D модель кольца, но заставить её повернуться в сторону Солнца
- Например, взять направление из Солнца на камеру за ось Y , взять любые две перпендикулярных ей и друг другу оси X и Z , и построить из них матрицу поворота
- Для динамического свечения можно тоже динамически обновлять буфер с цветами



Звёзды

- Для звёзд лучше использовать модель попроще (например, октаэдр или низкополигональную сферу)
- Все звёзды должны рисоваться одной командой и использовать instancing: нужен отдельный буфер с инстансами звёзд (например, положение, цвет, размер), и правильная настройка render pipeline
- Цвета звёзд можно взять случайными, а можно взять из формул blackbody spectrum :)
- Позиции звёзд можно сделать случайными единичными векторами
- Чтобы звёзды казались бесконечно далеко, рисуем их самыми первыми в кадре, без теста глубины и без записи в буфер глубины (это может быть отдельный render pass, или правильно настроенный pipeline), и используем для них особую view-матрицу камеры, в которой есть поворот, но нет сдвига на позицию камеры



- **1 балл:** рисуются хоть какие-то сферы
- **1 балл:** модель сферы – manifold mesh
- **1 балл:** камера двигается по WASD
- **1 балл:** камера крутится мышью
- **1 балл:** есть центральное Солнце и планеты вокруг него
- **1 балл:** есть спутники
- **1 балл:** есть кольца
- **1 балл:** все вращаются вокруг своей оси
- **2 балла:** у Солнца анимируются цвета
- **1 балл:** есть звёзды
- **2 балла:** все звёзды рисуются одним instanced draw
- **2 балла:** У Солнца есть “свечение”

Всего: **15 баллов**

Защита заданий на практике **через 2 недели.**